

positivity tactic

Pan Lin

June 18, 2026

What positivity proves

```
example (x : ℤ) : 0 ≤ x ^ 2 := by
  positivity
```

```
example (x : ℤ) : 0 < x ^ 2 + 3 := by
  positivity
```

```
example (x : ℤ) (hx : x ≥ 12) : x ≥ 0 := by
  positivity
```

```
example (x y : ℤ) (h : x > y) : 0 < max (-1) (x - y) := by
  positivity
```

```
example : (3 : ℤ) ≠ 0 := by
  positivity
```

```
example {a b : ℤ} (hab : 0 < a * b) (hb : 0 ≤ b) : 0 < a := by
  positivity [pos_of_mul_pos_left hab hb]
```

- solves goals of the form $0 \leq x$, $0 < x$ and $x \neq 0$.
- recursively analyze the positivity of the expression x ,
- attempt to prove subexpressions are positive/nonnegative/nonzero
- combine all the strictness into a final proof.
- either closes the goal or fails.

What positivity proves

```
example (x : ℤ) : 0 ≤ x ^ 2 := by
  positivity
```

```
example : (3 : ℤ) ≠ 0 := by
  positivity
```

```
example (x : ℤ) : 0 < x ^ 2 + 3 := by
  positivity
```

```
example (x : ℤ) : 0 ≤ x ^ 2 :=
  x.sq_nonneg
```

```
example : (3 : ℤ) ≠ 0 := by
  norm_num
```

```
example (x : ℤ) : 0 < x ^ 2 + 1 :=
  add_pos_of_nonneg_of_pos x.sq_nonneg (by norm_num)
```

How positivity proves

current goal

↓

Positivity.positivity : MVarId → MetaM Unit

reads the target proposition

↓

Positivity.solve : Q(Prop) → MetaM Expr

accepts: $0 < e$, $0 \leq e$, $e \neq 0$, $0 \neq e$

extracts: a , za , $pa?$, e , desired result

otherwise: "not a positivity goal"

↓

Positivity.core za $pa?$ e : MetaM (Strictness za e $pa?$)

recurse on the structure of e to compute the Strictness

returns: positive / nonnegative / nonzero

throws if the final result is none

↓

Positivity.solve catches core failure as none and checks the result

positive closes $0 < e$, $0 \leq e$, $e \neq 0$, $0 \neq e$ goals

nonnegative closes $0 \leq e$ goals

nonzero closes $e \neq 0$, $0 \neq e$ goals

↓

goal.assign proof

Positivity.Strictness

```
variable {u : Level} {a : Q(Type u)} (za : Q(Zero $a))

/-- The result of `positivity` running on an expression `e` of type `a`. -/
inductive Strictness (e : Q($a)) : Option Q(PartialOrder $a) → Type where
| positive {pa : Q(PartialOrder $a)} (pf : Q(0 < $e)) : Strictness e pa
| nonnegative {pa : Q(PartialOrder $a)} (pf : Q(0 ≤ $e)) : Strictness e pa
| nonzero {pa?} (pf : Q($e ≠ 0)) : Strictness e pa?
| none {pa?} : Strictness e pa?
```

- › **positive**: proof of $0 < e$
- › **nonnegative**: proof of $0 \leq e$
- › **nonzero**: proof of $e \neq 0$
- › **none**: no useful fact found
- › $Q(T)$: an Expr known to have type T
- › $q(t)$: quote the Lean term t as an Expr
- › $\$e$: splice an existing $e : \text{Expr}$ into a quotation

Inside Positivity.core

```
input:
  ↓
  za : Q(Zero $a)
  pa? : Option Q(PartialOrder $a)
  e   : Q($a)
  ↓
result := none
  ↓
registered positivity extensions
  ↓ result ← orElse result <| ...
  ↓
norm_num
  ↓ result ← orElse result <| ...
  ↓
canonical nonnegativity
  ↓ result ← orElse result <| ...
  ↓
compare with each local hypothesis
  ↓ result ← orElse result <| ...
  ↓
throw an error if the result is none, otherwise return it
  ↓
```

orElse	none	$\leq$	$\neq$	$<$
none	none	$\leq$	$\neq$	$<$
$\leq$	$\leq$	$\leq$	$<$	$<$
$\neq$	$\neq$	$<$	$\neq$	$<$
$<$	$<$	$<$	$<$	$<$

Compare with a local hypothesis

input:



expression e

local hypothesis h

hypothesis

extra evidence

returned proof

$h : 0 \leq e$

$h : 0 < e$

nonnegative h

positive h

$h : lo \leq e$

$h : lo \leq e$

$p_1 : 0 < lo$

$p_1 : 0 \leq lo$

positive (lt_of_lt_of_le p_1 h)

nonnegative (le_trans p_1 h)

$h : lo < e$

$h : lo < e$

$p_1 : 0 < lo$

$p_1 : 0 \leq lo$

positive (lt_trans p_1 h)

positive (lt_of_le_of_lt p_1 h)

$h : x = e$

$h : x = e$

$h : x = e$

$p_1 : 0 < x$

$p_1 : 0 \leq x$

$p_1 : x \neq 0$

positive (lt_of_lt_of_eq p_1 h)

nonnegative (le_of_le_of_eq p_1 h)

nonzero (ne_of_ne_of_eq' p_1 h)

$h : e \neq 0$

$h : 0 \neq e$

nonzero h

nonzero h .symm

A run through positivity

```
set_option trace.Tactic.positivity true in
example (x : ℤ) (hx : x ≥ 12) : x ≥ 0 := by
  positivity
```

1. solve reads the goal
target: $x \geq 0$
relation desired: $\leq$
2. core starts on the expression
 $e := x$
result := none
3. compare with hypotheses
 $hx : x \geq 12$
4. compareHyp matches the hypothesis shape: $lo \leq e$
 $lo := 12$
 $e := x$
5. compareHyp checks the lower bound by norm_num
 $0 < 12$
so lo is positive
6. compareHyp returns a stronger result
positive x
proof: lt_of_lt_of_le ($_ : 0 < 12$) hx
7. solve checks the result

```
desired ≤  
returned <  
goal closes
```

A run through positivity

```
set_option trace.Tactic.positivity true in
example (x : ℤ) : 0 < x ^ 2 + 3 := by
  positivity
```

1. solve reads the goal
target: $0 < x^2 + 3$
relation desired: $<$
2. core starts on the expression
 $e := x^2 + 3$
result := none
3. the addition extension applies
core (x^2) gives $\leq$
core 3 gives $<$
4. the two child results are combined by evalAdd
 $\leq$ and $<$ gives $<$
5. solve checks the result
desired $<$

returned <
goal closes

positivity is extensible

```
-- An extension for 'positivity'. -/  
structure PositivityExt' where  
  -- Attempts to prove an expression 'e : a' is '>0', '≥0', or '≠0'. -/  
  eval {u : Level} {a : Q(Type u)} (za : Q(Zero $a)) (pa? : Option Q(PartialOrder $a))  
    MetaM (Strictness za e pa?)
```

One extension handles one outer shape.

- inspect the outer expression pattern
- select the relevant children
- call **core** recursively
- choose a theorem
- build **Strictness** e

Extension lookup

```

/-- The `positivity` extension which identifies expressions of the form `a + b`,
such that `positivity` successfully recognises both `a` and `b`. -/
@[positivity _ + _] def evalAdd : PositivityExt where eval {u a} za pa? e := do
  let .app (.app (f : Q($a → $a → $a)) (a : Q($a))) (b : Q($a)) ← whnfR e | throwError "not +"
  let _e_eq : $e =Q $f $a $b := {}
  let _a ← synthInstanceQ q(AddZeroClass $a)
  assumeInstancesCommute
  let (<_f_eq>) ← withDefault <| withNewMCTxDepth <| assertDefEqQ q($f) q(HAdd.hAdd)
  let ra ← core za pa? a; let rb ← core za pa? b
  let some _pa := pa? | pure .none
  match ra, rb with
  | .positive pa, .positive pb =>
    let _a ← synthInstanceQ q(AddLeftMono $a)
    pure (.positive q(add_pos' $pa $pb))
  | .positive pa, .nonnegative pb =>
    let _a ← synthInstanceQ q(AddLeftMono $a)
    pure (.positive q(add_pos_of_pos_of_nonneg $pa $pb))
  | .nonnegative pa, .positive pb =>
    let _a ← synthInstanceQ q(AddRightMono $a)
    pure (.positive q(Right.add_pos_of_nonneg_of_pos $pa $pb))
  | .nonnegative pa, .nonnegative pb =>
    let _a ← synthInstanceQ q(AddLeftMono $a)
    pure (.nonnegative q(add_nonneg $pa $pb))
  | _, _ => failure

```

- › positive + positive -> positive
- › positive + nonnegative -> positive
- › nonnegative + positive -> positive
- › nonnegative + nonnegative -> nonnegative

Branching on `pa?`

When `pa? = .none`:

- Possible results:
 - `nonzero pf`
 - `none`
- Still useful for:
 - local `nonzero` hypotheses
 - multiplication with `NoZeroDivisors`
 - examples over $\mathbb{C}$

```
@[positivity _ ^ (0 : ℕ)]
meta def evalPowZeroNat : PositivityExt where eval {u a} _za pa? e := do
  let .app (.app _ (a : Q($a))) _ ← whnfR e | throwError "not ^"
  let _a ← synthInstanceQ q(Semiring $a)
  assumeInstancesCommute
  haveI' : $e =Q $a ^ 0 := ⟨⟩
  let _a ← synthInstanceQ q(Nontrivial $a)
  let some _pa := pa? | pure (.nonzero q(pow_zero_ne_zero $a))
  let _a ← synthInstanceQ q(IsOrderedRing $a)
  pure (.positive q(pow_zero_pos $a))
```

Repeated wrappers

expression seen by core	corresponding theorem
<code>n.succ</code>	<code>Nat.succ_pos n</code>
<code>n.factorial</code>	<code>Nat.factorial_pos n</code>
<code>p : PNat</code>	<code>PNat.pos p</code>
<code>q.den</code>	<code>Rat.den_pos q</code>

Same wrapper pattern:

- recognize the outer expression
- read the arguments
- apply the theorem
- wrap the result as `Strictness`

```
@[positivity Nat.succ _]
meta def evalNatSucc : PositivityExt where eval {u a} _za pa? e := do
  let some _ := pa? | throwError "not PartialOrder  $\mathbb{N}$ "
  match u, a, e with
  | 0, ~q( $\mathbb{N}$ ), ~q(Nat.succ $a) =>
    assertInstancesCommute
    pure (.positive q(Nat.succ_pos $a))
  | _, _, _ => throwError "not Nat.succ"
```

Work in Progress: positivity_lemma

```
attribute [positivity_lemma]
```

```
  Nat.succ_pos
```

```
  Nat.factorial_pos
```

```
  Rat.den_pos
```

```
Nat.succ_pos n : 0 < n.succ
```

```
registered data:
```

```
  key      Nat.succ / 1 argument
```

```
  result   <
```

```
  premises none
```

The attribute records how a theorem can be used by `core`.

Lemma shape

accepted conclusions:

$$0 < f \ x_1 \ \dots \ x_n$$

$$0 \leq f \ x_1 \ \dots \ x_n$$

$$f \ x_1 \ \dots \ x_n \neq 0$$

$$f \ x_1 \ \dots \ x_n > 0$$

$$f \ x_1 \ \dots \ x_n \geq 0$$

$$0 \neq f \ x_1 \ \dots \ x_n$$

accepted premises:

$$0 < x_i$$

$$0 \leq x_i$$

$$x_i \neq 0$$

from the conclusion:

head symbol

arity

result kind

from each premise:

argument index

required strictness

Applying a lemma: no premises

```
example (n : ℕ) : 0 < n.succ := by
  positivity
```

1. core starts on the expression
e := n.succ
result := none
2. lookup registered lemmas
key: Nat.succ / 1 argument
3. candidate lemma
Nat.succ_pos
premises: none
4. build the proof
Nat.succ_pos n
5. return Strictness
positive n.succ

Applying a lemma: with premises

```
example (q : ℚ) (hq : 0 < q) : 0 < q.num := by
  positivity
```

1. core starts on the expression
e := q.num
key: Rat.num / 1 argument
2. candidate lemma
num_pos_of_pos : 0 < q → 0 < q.num
3. the premise is registered as
argument 0 must be <
4. prove the premise by a recursive call
core q gives <
proof: hq
5. apply the theorem
num_pos_of_pos hq

```
6. return Strictness
   positive q.num
```

Back inside Positivity.core

```
result := none
  ▼
registered positivity extensions
  | result ← orElse result <| ...
  ▼
registered positivity lemmas
  | result ← orElse result <| ...
  ▼
norm_num
  | result ← orElse result <| ...
  ▼
canonical nonnegativity
  | result ← orElse result <| ...
  ▼
compare with each local hypothesis
  | result ← orElse result <| ...
  ▼
throwNone result
```

Future Plan

- Replace more mechanical PositivityExts with `@[positivity_lemma]`
- A `PositivityM` monad for cache/logging/fuel
- Success cache for repeated recursive goals
- Add support for local hypothesis search, so theorem-generated rules can discharge premises from assumptions

Speaker notes